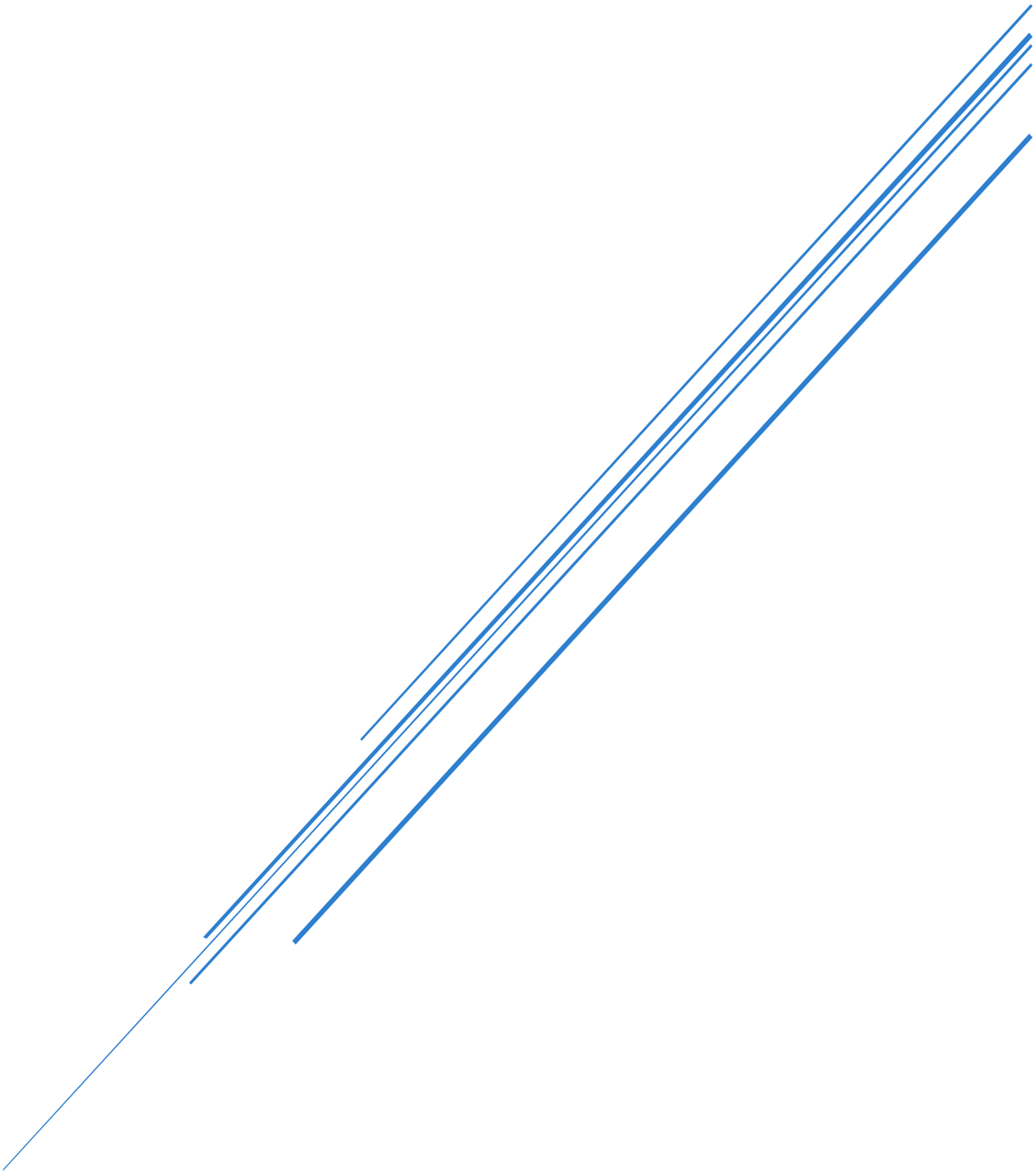
DETECTION MODEL COMPARISON

MMA3001 Project



Jared Ruberu - 33854920
Monash University

Engineering Problem

In this project, I have chosen to explore how machine learning techniques can be used to automate food packaging quality inspection processes. Packaging defects in commercial food retail can cause waste, consumer complaints, and recalls, whilst unsealed packaging may also affect food safety and shelf life. Manual inspection of food packaging can be slow, inconsistent, and fatiguing, limiting its ability to scale with the growing rate of production line speeds. Therefore, implementation of automated detection models could flag defective trays before they are distributed, minimising the negative impacts of packaging errors and reducing human effort and error.

This project aims to narrow the scope of this issue by comparing two detection methods, YOLOv8n and Faster R-CNN ResNet-50 FPN detection models, to uncover the strengths and weaknesses of each model to determine which option offers the best trade-offs for real-time inspection. The factors which will be considered are accuracy, speed, and resource use of each model.

The intended use of this solution is to provide a benchmark to support decision making regarding detector models for inspection, should companies be considering implementing one into their own production systems. This research could also be used in the creation of automated pass/fail systems for industrial production, further utilising the capabilities of modern computing to minimise human effort.

It should be considered that the scope of this research also has limitations. Only considering two models restricts the ability to be able to comment on overall trends and themes, as well as model-size scaling or different methods such as transformer models. Additionally, the dataset in use is of the single source variety, using a consistent tray and packaging type, with a single stationary camera setup. Therefore, results may not transfer to other lighting, tray designs, or products.

Inputs and Outputs

Inputs:

Images: The image dataset used was sourced via a public Roboflow listing, containing over 2000 8-bit RGB images sized 720x540 pixels. The images were later resized to 640x640 pixels to be compatible with the YOLOv8n model. For this project, a random generated 200 images were selected due to limitations in computing and time available. The models selected automatically resize images before processing.

Annotations: Annotations were provided in a YOLO text format, describing the type of packaging error for validation and testing purposes.

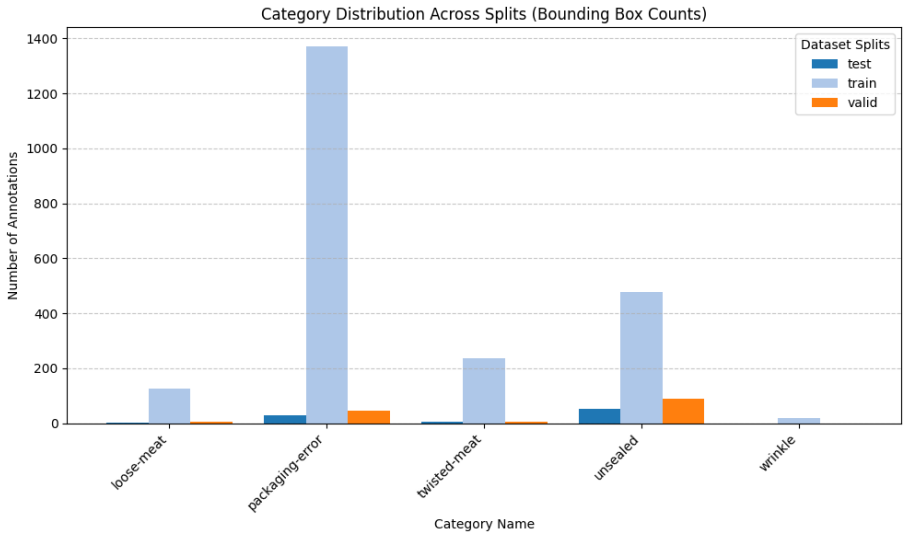


Figure 1: Distribution of full dataset error categories

Outputs:

Per-Image detections: Detections are in the form of boxes using pixel coordinates, with class labels and a confidence score between 0 and 1.

Accuracy Metrics: mAP@0.5, mAP@0.5:0.95, precision metrics, per-class AP, and a confusion matrix. These datapoints assist with comparing models in terms of accuracy and locating weak classes.

Speed Metrics: ms per image and throughput FPS used to measure how long each model takes to process images.

Resource Metrics: Parameter Count, model file size, GFLOPs, peak GPU memory, and training time used to judge deployment cost of each model.

Computational Solution

Workflow Overview:

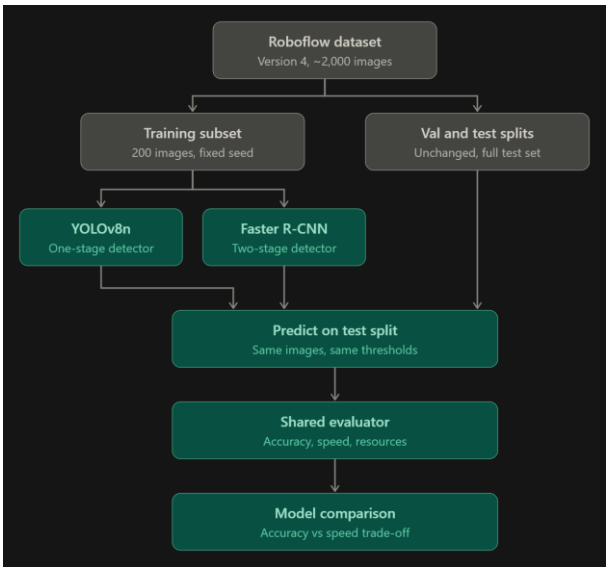


Figure 2: Workflow diagram (generated by Claude AI)

Workflow in this project involves using a public Roboflow dataset as the inputs for the machine learning models. Initially, a quick verification process was created to test that the images and annotations were valid and able to be processed, and any items that returned a null reading was removed. The dataset was then reduced to a 200-image subset utilising a random seed that was consistent between both models. The models were then both trained on the same training subset and tested against the same validation data, using the same evaluation code and thresholds. As models have been trained on the same data, comparisons can be drawn between their accuracy, speed, and resource requirement.

Model Overview:

YOLOv8n: a single stage detector which uses a single network pass to predict boxes and classes from the whole image. Features a convolutional backbone which extracts features, a neck which combines features on several scales, and a detection head that predicts box and class scores on each location. This model creates many overlapping predictions; therefore, duplicates are removed and the highest confidence boxes are kept via non-maximum suppression (NMS).

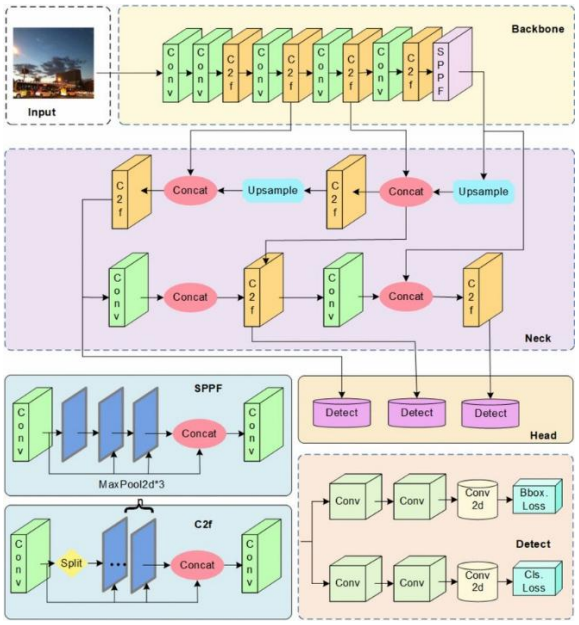


Figure 3: Block diagram visualising Yolov8n process (source: https://www.researchgate.net/figure/YOLOv8n-architecture-diagram-The-YOLOv8n-architecture-consists-of-three-main-components_fig1_395971217)

Faster R-CNN ResNet-50 FPN: A two-stage model. Stage one involves a ResNet-50 backbone with a feature pyramid network (FPN) which extracts multi-scale features. A region proposal network (RPN) slides over these features and proposes regions likely to contain and object using a set of predefined anchor boxes. Stage two involves each proposed region being pooled to a fixed size, then a head classifies it and refines the box coordinates. Similarly, an NMS process is undertaken to minimise overlap.

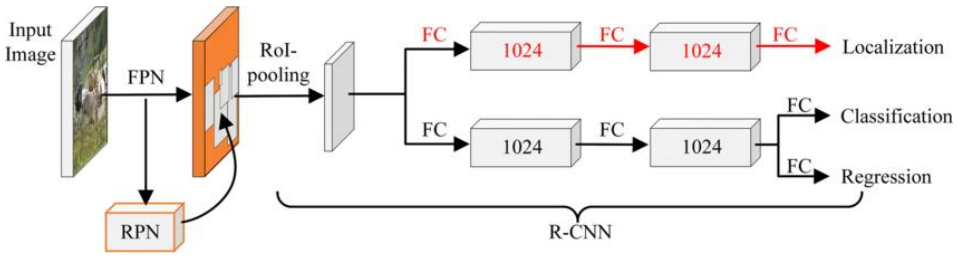


Figure 4: Block diagram visualising Faster R-CNN ResNet-50 FPN model (source: https://www.researchgate.net/figure/Faster-R-CNN-model-with-ResNet-50-and-FPN-The-proposed-localization-confidence_fig4_371671063)

Settings and Parameters

Parameter	Value
No. of Images	200
Image Size	640x640 pixels
Epochs	20 for both models
Batch Size	16 for both models
Confidence Threshold	0.25
NMS IoU Threshold	0.45
Hardware	Google Colab GPU Runtime

Validation

Validation was implemented into the computational solution to ensure that the results of the experiment could be considered trustworthy and reliable. Evidence of validation can be considered across three areas of the solution: verification, detection performance, and speed/resource measurements.

Verification (Code and Data)

Prior to conducting the experiment, the dataset was checked over and corrected for data leakage. Identical copies of photos in each of the split were recognised and removed from the datasets, protecting the validity of the experiment. Similarly, when the dataset was minimised into a 200-image subset, the subset manifest, hash, and pytest checks were constructed to ensure that the fixed dataset consisted of 200 unique images which remained consistent across all parts of the experiment. The evaluation processes were similarly ensured to remain consistent in their confidence leveling and NMS settings for both models, ensuring that differences in the models come from only the models themselves.

Detection

Validating the detection metrics came in the form of ensuring consistent standards and settings were kept across both model testing. For both models, test data was consistent and withheld from training processes, ensuring that training was conducted on unseen images, and that the annotations provided with the test split were kept for validation as the ground truth.

Speed/Resource measurements

Similarly, consistency was kept throughout the speed and resource comparison settings and metrics to ensure a true and accurate representation of the differences of the models compared to each other. Settings such as epoch count, batch size, and GPU usage were the same across both models, and sanity checks were put in place such as latency and FPS comparisons to ensure that the data output by the model measurements were consistent with expectations.

Results

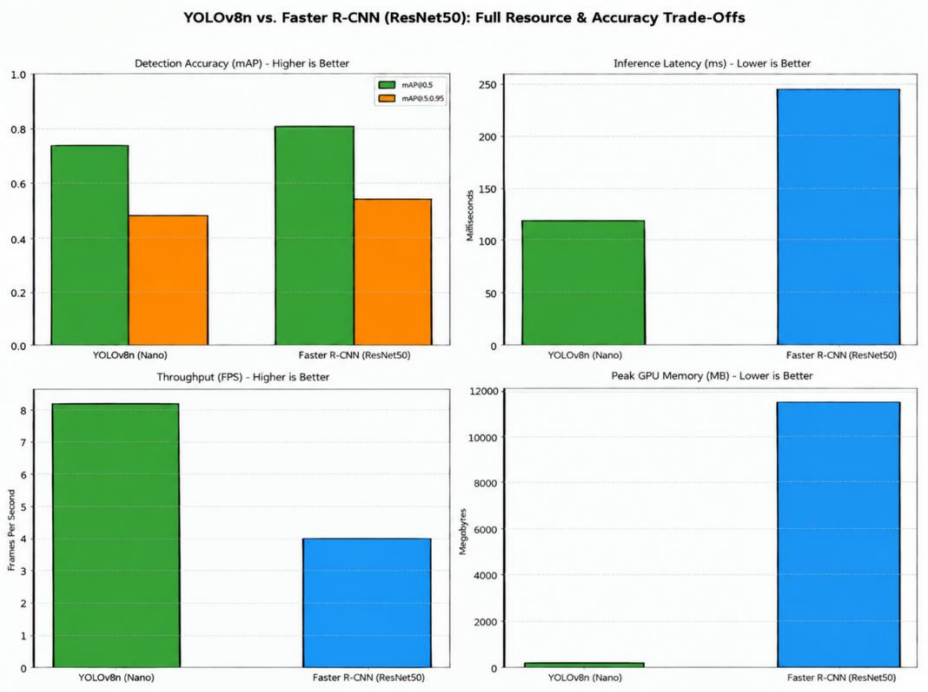


Figure 5: Results comparison between models

Metric	YOLOv8n	Faster R-CNN
Accuracy (mAP@0.5)	0.712	0.812
Accuracy (mAP@0.5:0.95)	0.485	0.541
Inference Latency (ms)	120.63	247.561
FPS	8.289	4.039
Peak GPU (mb)	*BUGGED*	11523.500

Accuracy: The Faster R-CNN model scores higher in accuracy compared to YOLO in both the mAP@0.5 and mAP@0.5:0.95 accuracy comparisons. This is to be expected, as the larger two-stage model is more fine-tuned and therefore will usually perform better in accuracy compared to a single-step nano model on a small dataset. Therefore, the R-CNN model is better for producing more accurate detections.

Speed: Based on the data derived from the experiment, the YOLOv8n model operates with a lower latency, and thus a higher FPS, suggesting that the single step model is faster than the two step model of R-CNN. Therefore, the YOLO model is better for more quickly detecting objects.

Resource use: During the experiment, the recording of peak and allocated GPU memory for the YOLOv8n model bugged and did not record, indicating a limitation in the solution. However, it is expected that the YOLO model will demonstrate superior resource consumption metrics compared to R-CNN. This is due to the YOLO model having fewer parameters and operations, and it being a single pass model. These factors result in the overall size of the architecture and the computing requirements of

the YOLO model being lesser than the R-CNN model. Future experiment is still recommended to confirm this statement and to obtain a numerical value for GPU usage to be able to use for a more detailed comparison.

Overall verdict: The trade-off between the YOLO and R-CNN model is that the R-CNN model is more accurate at the cost of processing speed and a higher resource consumption. Companies which have access to better hardware and are more conscious of accurately detecting defects might be more aligned with selecting the R-CNN model as the detection model, given that they can afford to slow the production line to account for the slower detection processing speed. However, the YOLOv8n model emerges as the more suitable model for basic commercial use due to the lesser associated resource cost, more efficient speed and processing, and only slightly decreasing in accuracy in comparison.

AI Usage and Reflection

AI was utilised as a tool to enhance productivity and aid in the completion of the project with human supervision throughout all sections of the assignment. AI tools such as Claude AI and Google Colab inbuilt Gemini were employed in the creation of the project. Throughout the project planning, AI tools were utilised to assist project selection, dataset comprehension, and shortlisting possible detectors to explore and compare.

Throughout the coding segments, the majority of the code was generated via AI models, particularly the model-specific coding functions and metric generators. AI code generation proved a valuable asset as it mitigated the need to extensively research how to structure coding programs, especially as the models used were new coding concepts to me. All code generated was looked over and checked by a human, and the AI models were instructed to include in-line comments and docstrings to ensure newcomers can follow along with what processes were being modelled by which lines of code. Additionally, the included sanity checks in the validation sections of the project ensured that AI generated content behaved as expected, further confirming the legitimacy of the code generated.

AI was used to assist in the structure of the written report and to make brief notes of what to include in each section, but all written content was created and written by myself. AI was also used to generate certain diagrams throughout the report, in which the use of AI was acknowledged in the figure. AI also valuable in reinforcing my own understanding of relevant computer concepts to ensure that the analysis of the results was correct and that discussions were valid and informative.

AI occasionally generated code, which was erroneous on first generation, resulting in programs which didn't operate correctly or resulted in failures when running. All code had to be run initially to ensure functionality met expectations, and debugging was required on certain occasions. This highlighted the importance of checking that code acts as anticipated when generated before pushing the code or moving on, as it cannot

always be trusted that the AI generation achieved what was expected on the first attempt.